BACKEND DE APLICACIONES · 1º PARCIAL

Guía única del 1º parcial

El parcial real, el preparcial, los cuestionarios y los simulacros: todas las preguntas sin repetir, con la respuesta correcta y el porqué

Parcial real 3K1 tal cual | Con la corrección del profe | Solución de la parte práctica | 79 preguntas | Verificado corriendo el código | Errores marcados

Qué hay en esta guía

Junta en un solo lugar, **sin repetidas**, todo lo que hay para el 1º parcial: el **parcial real** (3K1), preguntas sobre el **proyecto del preparcial**, el **Cuestionario 05** (Streams), la **teoría** de los apuntes y cuestionarios, y las preguntas de los **cuatro simulacros**. La base es la guía de repaso que circulaba (la más completa), corregida. Se le agregó lo que faltaba de los simulacros y del parcial, y se sacaron las repetidas: si una pregunta salía igual en dos lados, va una sola vez, con el origen indicado.

- Opción correcta (✓)
- Respuesta modelo (desarrollo)
- "De práctica": armada sobre el código o el apunte, no salió así en el campus
- Por qué
- Error del campus, de la guía o del enunciado

Correcciones respecto de la guía de repaso

- Streams, pregunta "referido a Stream API": tiene **dos** correctas, no una (sección 3).
- maximaTasaInteresPorTipo: ahora está resuelta con el código del simulacro (b, por descarte; sección 5).
- srv-0006 es inválido por CPU, no por índice (sección 2). Se agregaron las preguntas del cuestionario del simulacro que faltaban (lógica de negocio, características de Java, sobrecarga, redefinición, persistencia).

Índice

1. El parcial real · 3K1 · 19/09/2026
2. El proyecto del preparcial (Cloud Center)
3. Stream API y programación funcional (Cuestionario 05)
4. Teoría: backend, Java, POO, Maven, excepciones, JUnit y CSV
5. Práctica con los proyectos de los simulacros

1 El parcial real · 3K1 · 19/09/2026

Así fue el 1º parcial (3K1, sábado 19/09/2026)

- **El día anterior** dieron el **preenunciado** y el proyecto **01-modelo-inicial** (Cloud Center) para estudiarlo y ejecutarlo. No había que programar nada: había que entenderlo.
- **El día del parcial**: un cuestionario de 10 preguntas en el campus. La **1** (15 puntos) era subir el ZIP del proyecto modificado según el **enunciado** (unos 45 minutos de trabajo), corregido a mano con una rúbrica sobre 10. Las **2 a 4** (2 puntos cada una) eran sobre el código del proyecto. Las **5 a 9** (1 punto cada una) eran teoría que sale de los cuestionarios semanales. La **10** (5 puntos) era un desarrollo corto sobre el diseño.
- Cómo se reparten los 31 puntos:

  

  Casi la mitad es el código: por eso conviene asegurarlo primero (ver "Cómo corrige el profe la parte práctica").
- Los cuatro simulacros (3K1 fintech, 3K3 taller, 3K4 juegos, 3K7 tarjetas) siguen el mismo esquema: preenunciado, enunciado y tests nuevos.

Abajo están las preguntas **tal cual salieron** (captura del campus, con la respuesta que marca el campus). Debajo de cada una va la respuesta verificada y el porqué.

Pregunta 1 · 15 puntos · Parte práctica: "Ingreso promedio por Servidor"

"Adjunte un único ZIP del proyecto resuelto (pom.xml, src/, datos/ y DECISIONES.md). Excluya target/, .git/ y archivos del IDE. Se califica manualmente con la rúbrica práctica sobre 10 puntos."

Lo que pedía el enunciado (partiendo del preparcial, sin romper sus tests):

- Nuevo encabezado `id,region,porcentajeCPU,usoMemoria,estado,ingresos`. El formato viejo de 5 columnas **sigue valiendo**; ahí el ingreso es 1000 como antes.
- Cada fila tiene que tener el ancho de **su** encabezado: no se aceptan filas de 5 campos con un encabezado de 6, ni al revés.
- En ALERTA o CRITICO del formato nuevo, ingresos tiene que cumplir **1000 ≤ ingresos < 1900** (después del strip). Si no, la fila es inválida.
- `costoRiesgoOperativo()`, `total()`, los filtros y los totales por región usan el ingreso de la fila.
- Agregar al colector (`Cloud`) un **conteo por estado** (vacío o en cero si no hay servidores).
- El main lee por defecto `datos/datos-parcial.csv`. Se entrega el ZIP más un `DECISIONES.md` de 3 a 5 líneas.

El ejemplo del enunciado tiene un error de cuenta

Dice que con índice 65 e ingreso 1300 el costo es 877 USD, pero 1300 × 65 / 100 = **845**. Con 877 el ingreso tendría que ser 1350. Los tests del profe (`EvolucionTest`) confirman la fórmula **ingresos × índice / 100**: srv-0054 (índice 59, ingresos 1110) da 654 y srv-0123 (índice 53, ingresos 1253) da 664.

Cómo se resolvía (solución propuesta, pasa los 12 tests: los 10 viejos y los 2 nuevos)

Es la jerarquía que sugiere la pregunta 10: `ServidorConIngresos extends Servidor`. El cálculo queda en un solo lugar y la subclase cambia solo el ingreso.

1. **Servidor**: el costo usa un método que la subclase puede redefinir, y además guarda el estado (hace falta para contar por estado).

```
protected int ingresoPorHora() { return INGRESO_PROMEDIO; }        // 1000

public int costoRiesgoOperativo() {
    return (this.ingresoPorHora() * this.indiceCarga) / 100;        // multiplicar antes de dividir
}

public static Servidor desdeCampos(String[] tokens) {
    if (tokens.length != 5 && tokens.length != 6) throw new IllegalArgumentException("se esperan 5 o 6
columnas");
    ...   // mismas validaciones de cpu, memoria e índice que antes
    if (tokens.length == 6)
        return new ServidorConIngresos(tokens[0], tokens[1], indiceCarga, tokens[4],
Integer.parseInt(tokens[5]));
    return new Servidor(tokens[0], tokens[1], indiceCarga, tokens[4]);
}
```

2. **ServidorConIngresos**: valida su dato en el constructor y redefine el ingreso.

```
public class ServidorConIngresos extends Servidor {
    private final int ingresos;
    public ServidorConIngresos(String id, String region, int indiceCarga, String estado, int ingresos)
{
        super(id, region, indiceCarga, estado);                 // la base valida id, región e índice
        if (ingresos < 1000 || ingresos >= 1900) throw new IllegalArgumentException("los ingresos estan
fuera de rango");
        this.ingresos = ingresos;
    }
    @Override
    protected int ingresoPorHora() { return ingresos; }
}
```

3. **ParserServidores**: una sola carga. El encabezado dice cuántas columnas se esperan y cada fila se controla contra ese número.

```
String cabecera = lector.readLine();
int columnas;
if (CABECERA.equals(cabecera)) columnas = 5;
else if ((CABECERA + ",ingresos").equals(cabecera)) columnas = 6;
else throw new IllegalArgumentException("encabezado incorrecto");
...
if (tokens.length != columnas) throw new IllegalArgumentException("cantidad de columnas incorrectas");
```

4. **Cloud**: el conteo por estado. `total()`, `totalesPorRegion()` y `filtrar()` **no se tocan**: ya llaman a `costoRiesgoOperativo()` y el polimorfismo hace el resto.

```
public Map<String, Long> conteoPorEstado() {
    return servidores.stream()
            .collect(Collectors.groupingBy(Servidor::getEstado, TreeMap::new, Collectors.counting()));
}
```

5. **App**: archivo por defecto `datos/datos-parcial.csv` y una línea con **ALERTA y CRITICO por separado** (en cero si no hay: el profe pide las dos cantidades). Salida con el archivo del parcial:

```
Map<String, Long> porEstado = colector.conteoPorEstado();
System.out.println("ALERTA: %d | CRITICO: %d".formatted(
        porEstado.getOrDefault("ALERTA", 0L), porEstado.getOrDefault("CRITICO", 0L)));
```

```
Leídas: 140 | procesadas: 49 | descartadas: 70 | inválidas: 21 | objetos: 49
ALERTA: 49 | CRITICO: 0
Total: 35800 | Por Region: {SA-EAST-1=12085, US-EAST-1=12712, US-WEST-2=11003} | indice menores a 50:
26
```

Ningún CRITICO llega a procesarse: los 12 del archivo tienen CPU o memoria fuera de rango, o un índice mayor a 70. La línea 128 es ALERTA con ingresos vacío, así que es inválida (`For input string: ""`).

Cómo corrige el profe la parte práctica

La devolución del campus muestra el método: **corre estos comandos en orden** sobre el ZIP entregado y después revisa el enunciado requisito por requisito.

#	Comando	Qué mira
1	`mvn clean`	Que el proyecto se pueda limpiar (el pom.xml está bien).
2	`mvn compile`	Que el código principal compile.
3	`mvn test`	Que compilen y pasen **los tests viejos y los nuevos**. Si los tests no compilan (por ejemplo, porque se cambió la firma de un constructor que usan), **no corre ninguno** y es "falla funcional".
4	`java -cp target/classes ar.edu.backend.App datos/datos.csv`	Que el CSV **viejo** siga dando lo mismo que antes (30 servidores procesados).
5	`java -cp target/classes ar.edu.backend.App datos/datos_parcial.csv`	Que el CSV **nuevo** cargue objetos y que la salida muestre lo pedido. Ojo: lo corrió con **guion bajo**, como venía el archivo en el ZIP del enunciado.

Requisito que revisa	Qué tiene que pasar para que cuente
Encabezados de 5 y 6 columnas	El parser reconoce los dos y además exige que cada fila tenga el ancho de **su** encabezado.
Importar el CSV del parcial	Con el archivo nuevo se crean objetos (no cero).
Usar los ingresos en el costo	El parser **crea la subclase** y `costoRiesgoOperativo()` usa el ingreso de la fila. Una subclase que el parser nunca instancia no suma.
Mantener el formato original	El CSV viejo sigue procesando sus 30 servidores.
Contar estados y mostrarlos	Hay un conteo por estado en el colector **y** el main lo muestra. Cuidado con la condición: "ALERTA **y** CRITICO" con `&&` nunca se cumple para un mismo servidor; es `||`.
Expansión mediante herencia	La subclase existe **y** está conectada al flujo de importación (la fábrica `desdeCampos` la devuelve).
Main con el archivo del parcial	Por defecto lee el CSV del parcial y muestra resumen, cantidad de objetos, ALERTA y CRITICO **por separado**, total y totales por región.

Lo que más puntos hace perder

- **Romper los tests viejos.** No cambies ni borres constructores o métodos que usan los tests: agregá uno nuevo (sobrecarga) y dejá el viejo.
- **Subclase desconectada.** Crear `ServidorConIngresos` no alcanza: el parser o `desdeCampos` tienen que crearla cuando la fila tiene 6 campos.
- **El CSV viejo deja de andar.** Corré siempre los dos archivos antes de entregar.
- **Calcular pero no mostrar.** Si el main no imprime el conteo, el requisito no se cumple aunque el método exista.

El paso a paso para hacerlo en orden y asegurar puntos está en la página: Backend → 1º Parcial → “Paso a paso del parcial”.

Preguntas 2 a 10, tal cual salieron

Pregunta 2 del parcial — 2 puntos

Partiendo de la base que se importa en nuevo formato y las lineas que se importan son ("S1", "R1", 56, 4500, "ESTABLE"), ("S2", "R1", 56, 4500, "CRITICO", 1200), ("S2", "R1", 56, 34500, "CRITICO", 1200). Cuantas lineas cual seria el resultado esperado de la clase ResultadoParseo al ivocar al metodo informe?

Seleccione una:

- a. Leídas: 3 | procesadas: 1 | descartadas: 1 | inválidas: 1 | objetos: 1
- b. Leídas: 3 | procesadas: 1 | descartadas: 2 | inválidas: 0 | objetos: 1
- c. Leídas: 3 | procesadas: 0 | descartadas: 1 | inválidas: 1 | objetos: 0
- d. Leídas: 3 | procesadas: 2 | descartadas: 0 | inválidas: 1 | objetos: 2

Solo hay una sola linea procsada, ya que ESTABLE es descartado y S2 es invalida por superar el rango permitido de uso de memoria

La respuesta correcta es: Leídas: 3 | procesadas: 1 | descartadas: 1 | inválidas: 1 | objetos: 1

✓ Para el campus vale la **a**: Leídas 3 | procesadas 1 | descartadas 1 | inválidas 1 | objetos 1.

Ojo: Si lo corrés con el código del preenunciado, la fila 2 también es inválida: el índice da (int)(56·0,7 + 4500·0,3) = (int)(39,2 + 1350) = **1389** y el constructor de Servidor pide 1 a 70. El resultado real sería procesadas 0 | inválidas 2 | objetos 0, que no está entre las opciones (es el mismo caso que srv-0012 del preparcial). En el parcial elegí la que razona cada fila por el estado y los rangos de CPU y memoria, como el profe.

Por qué: Fila por fila, como razona el profe: (1) ESTABLE → **descartada**. (2) CRITICO, CPU 56 y memoria 4500 dentro de rango → **procesada**. (3) memoria 34500 > 24576 → **inválida**.

Pregunta 3 del parcial 2 puntos

Para el calculo del costo total por region se tiene el siguiente codigo

```
this.servidores.stream()
                .collect(Collectors.groupingBy(Servidor::getRegion,
                        Collectors.summingInt(Servidor::costoRiesgoOperativo)));
```

cual seria el Collectors correcto si quisiera obtener el costo promedio por region

Seleccione una:

- a. Collectors.averagingInt
- b. Collectors.summarizingInt
- c. Collectors.summarizingDouble
- d. Collectors.averagingDouble

Si bien es util el Collectors.summarizing... para este caso en particular sin necesidad de otro calculo estadistico lo mas correcto es usar Collector.averagingInt, ya que los datos que deseamos producir simpre son enteros

La respuesta correcta es: Collectors.averagingInt

✓ a. `Collectors.averagingInt`.

Por qué: `costoRiesgoOperativo()` devuelve `int`, así que el colector natural es `averagingInt(ToIntFunction)`, que devuelve el promedio como `Double`. `summarizingInt` también sirve, pero devuelve un `IntSummaryStatistics` (cantidad, suma, mínimo, máximo y promedio) y hay que sacar el promedio con `getAverage()`: es más de lo que se pide. `averagingDouble` compila (el int se convierte a double), pero no es el adecuado para datos enteros.

Pregunta 4 del parcial 2 puntos

En base a sus conocientos, Por qué el uso de la línea this.servidores = List.copyOf(servidores); dentro del constructor de una clase se considera una de las mejores prácticas de programación en Java

Seleccione una:

- a. Porque crea una copia defensiva e inmutable de la lista original, impidiendo que cambios externos sobre la colección original alteren el estado interno del objeto.
- b. Porque permite ordenar automáticamente los elementos de la colección al momento de asignarla, mejorando el rendimiento de las búsquedas posteriores.
- c. Porque convierte la lista en un arreglo primitivo dinámico de menor tamaño, reduciendo drásticamente el consumo de memoria en la máquina virtual (JVM)
- d. Porque realiza una copia profunda (deep copy) de cada uno de los objetos individuales contenidos dentro de la lista, clonando sus atributos de forma recursiva.

Esta práctica asegura la inmutabilidad y el encapsulamiento. Al utilizar List.copyOf, se genera una copia superficial (shallow copy) que queda blindada contra modificaciones externas. Si el código que invocó al constructor altera la lista original más adelante, el estado interno de nuestra clase permanece intacto y a salvo de efectos secundarios imprevistos. Además, la lista devuelta rechaza elementos null y arroja una excepción si se intenta modificar (UnsupportedOperationException)

La respuesta correcta es: Porque crea una copia defensiva e inmutable de la lista original, impidiendo que cambios externos sobre la colección original alteren el estado interno del objeto.

✓ a. Copia defensiva e inmutable.

Por qué: `List.copyOf` crea una lista nueva e inmodificable: si después alguien hace `clear()` sobre la lista original, el objeto no se entera. Ojo con la d: la copia es **superficial** (copia las referencias, no clona cada Servidor), y alcanza porque Servidor es inmutable (atributos `final`, sin setters). La lista copiada rechaza `null` y tira `UnsupportedOperationException` si se la intenta modificar.

Pregunta 5 del parcial 1 punto

¿Qué ventaja aporta la arquitectura hexagonal respecto a una arquitectura en capas tradicional?

- a. Fusiona todos los componentes en una única interfaz.
- b. Separa el dominio de la aplicación de las dependencias externas.
- c. Se enfoca únicamente en la persistencia de datos.
- d. Permite escribir menos código en el backend.

La respuesta correcta es: Separa el dominio de la aplicación de las dependencias externas.

✓ b. Separa el dominio de las dependencias externas.

Por qué: Es la misma pregunta del Cuestionario 01. En la arquitectura hexagonal (puertos y adaptadores) el núcleo con la lógica de negocio no depende de la base de datos, del framework web ni de las APIs externas: se conectan desde afuera mediante adaptadores.

Pregunta 6 del parcial 1 punto

El lenguaje de Programacion Java tiene un recolector de basura que se encarga de liberar la memoria ocupada por los objetos que no estan siendo utilizados

- a. Falso Incorrecto!
- b. Verdadero

La respuesta correcta es: Verdadero

✓ Verdadero.

Por qué: El **garbage collector** de la JVM libera la memoria de los objetos que ya no tienen referencias. Por eso en Java no se libera memoria a mano (no hay `free` ni `delete` como en C/C++).

Pregunta 7 del parcial 1 punto

¿Que significa que sobrecarguemos un constructor?

- [ ] a. Significa que el constructor haga mas que solo construir la clase.
- [x] b. Significa que hagamos multiples constructores para una misma clase. Correcto !
- [ ] c. Significa que el constructor tenga todos los parametros que puede tener.

La respuesta correcta es: Significa que hagamos multiples constructores para una misma clase.

✓ b. Hacer varios constructores para la misma clase.

Por qué: Sobrecargar es tener varios métodos (o constructores) con el **mismo nombre** y **distinta lista de parámetros**. En la solución del parcial pasa eso: `Servidor(id, region, indice)` y `Servidor(id, region, indice, estado)`.

Pregunta 8 del parcial 1 punto

Al usar el modificador final en una clase estamos diciendo que dicha clase no puede ser derivada y que al intentarlo nos sera entregado un error de compilacion

- ( ) Verdadero
- (•) Falso

La respuesta correcta es 'Verdadero'

✓ Verdadero (en la captura se ve marcado Falso, que está mal).

Por qué: Una clase `final` no se puede heredar: `class B extends A` con A final da **error de compilación**. Ejemplo: `String` es final. Un método `final` no se puede redefinir y una variable `final` no se puede reasignar.

Pregunta 9 del parcial — 1 punto

Seleccione las opciones correctas respecto a la operacion intermedia distinct?

- [ ] a. Ninguna es correcta
- [x] b. Retorna un nuevo flujo con los mismos datos del flujo original pero eliminando repetidos — Correcto
- [ ] c. Retorna una nueva lista con los mismos datos del flujo original pero eliminando repetidos

La respuesta correcta es: Retorna un nuevo flujo con los mismos datos del flujo original pero eliminando repetidos

✓ b. Devuelve un nuevo **flujo** sin repetidos.

Por qué: Es del Cuestionario 05 y salió igual. `distinct()` es una operación **intermedia**: devuelve otro `Stream`, no una lista. La trampa es la palabra "lista": lo mismo pasa con `limit`, `sorted`, `filter` y `map`.

Pregunta 10 del parcial — 5 puntos · desarrollo

Explique brevemente la ejecución del método costoRiesgoOperativo(). Teniendo en cuenta que podria existir una jerarquía de herencia simple compuesta por la clase base Servidor (que maneja atributos como CPU, memoria e índice de carga) y la subclase ServidorConIngresos (que extiende la base agregando un sexto campo numérico para los ingresos del servidor en estados de ALERTA o CRITICO).

Respuesta modelo: `costoRiesgoOperativo()` se define una sola vez en **Servidor**: (ingreso por hora × índice de carga) / 100, todo en enteros (se multiplica antes de dividir para no perder precisión). En la base el ingreso es la constante `INGRESO_PROMEDIO = 1000`. **ServidorConIngresos extends Servidor** agrega el sexto campo, `ingresos`. Su constructor llama primero a `super(...)`, que valida id, región e índice, y después valida que 1000 ≤ ingresos < 1900. Además **redefine** (`@Override`) el ingreso por hora. Cuando `Cloud` hace `mapToInt(Servidor::costoRiesgoOperativo)`, la lista es `List<Servidor>`, pero por **enlace dinámico** cada objeto ejecuta su versión: un Servidor común da 1000·índice/100 y uno con ingresos da ingresos·índice/100. Por ejemplo, srv-0054 (índice 59, ingresos 1110) da 65490 / 100 = **654** (la división entera trunca). Así `total()`, `totalesPorRegion()` y los filtros toman la regla nueva sin cambiar `Cloud`.

2 El proyecto del preparcial (Cloud Center)

Son preguntas armadas sobre el código **real** del proyecto (Servidor, ParserServidores, ResultadoParseo, Cloud). Todos los números salen de correr el programa y los tests. Así es como preguntan las 2 a 4 del parcial.

Lo que conviene tener en la cabeza del preparcial

- `índice = (int)(cpu·0,7 + memoria·0,3)`: el cast **trunca**. Tiene que quedar entre 1 y 70 (lo valida el constructor). CPU entre 1 y 100, memoria entre 1 y 24576.
- `costo = (1000 · índice) / 100`, en enteros.
- El orden del parser es: ancho → ESTABLE (se descarta sin validar nada más) → estado desconocido → `desdeCampos` (CPU, memoria, índice). Las inválidas se registran con "Linea N: motivo" y **la carga sigue**.
- Con `datos.csv`: 60 leídas, 30 procesadas, 21 descartadas, 9 inválidas. Total 15170. SA-EAST-1 = 5400, US-EAST-1 = 6790, US-WEST-2 = 2980. 16 con índice menor a 50.
- Casos testigo: srv-0044 → 50, srv-0021 → 50, srv-0045 → descarte ESTABLE, srv-0004 → estado desconocido (MANTENIMIENTO), srv-0012 → índice fuera de rango.

Error del preenunciado

Dice que srv-0006 es "inválido por índice de carga", pero srv-0006 (CPU **114**, memoria 4) es inválido por **CPU fuera de rango**: es el "Linea 7: uso del cpu esta fuera de rango" que controlan los tests. El caso de índice fuera de rango es srv-0012 (97, 16310 → 4960).

Pregunta 1
Una respuesta

Preparcial · Cloud Center

De práctica

¿Qué imprime este fragmento, sabiendo que Cloud copia la lista recibida con `List.copyOf` en su constructor?

```
var lista = new ArrayList<Servidor>();
lista.add(new Servidor("S1", "W1", 45));
var c = new Cloud(lista);
lista.clear();
System.out.println(c.total());
```

Seleccione una:

- a. 0
- b. 450 ✓
- c. Lanza NullPointerException
- d. Compila pero el resultado es indeterminado

La respuesta correcta es: 450

Por qué: Cloud guardó su propia copia antes del `clear()`, que vacía solo la lista original. El costo de un servidor con índice 45 es 1000·45/100 = 450. Es el test `copiaDefensiva` de CloudTest.

Pregunta 2
Una respuesta

Preparcial · Cloud Center

De práctica

Después de leer un archivo con ParserServidores se ejecuta `resultado.getServidores().add(new Servidor("X","R",10))`. ¿Qué ocurre?

Seleccione una:

- a. Se agrega sin problemas a la lista interna de ResultadoParseo
- b. Lanza UnsupportedOperationException ✓
- c. Lanza IllegalArgumentException
- d. Lanza ConcurrentModificationException

La respuesta correcta es: Lanza UnsupportedOperationException

Por qué: ResultadoParseo guarda sus tres listas con `List.copyOf`, que son inmodificables: `add`, `clear` o `remove` tiran `UnsupportedOperationException`. Lo prueba `resultadosYCopiasDefensivas`.

Pregunta 3
Desarrollo

Preparcial · Cloud Center

De práctica

¿En qué línea del proyecto se define el separador de columnas del CSV? Si hubiera que soportar archivos separados por `;`, ¿qué habría que cambiar?

```
String[] tokens = Arrays.stream(linea.split(",", -1))
        .map(String::strip)
        .toArray(String[]::new);
```

Respuesta modelo: En `ParserServidores.leer()`, en `linea.split(",", -1)`: es el único lugar donde está el separador. Para usar `;` hay que cambiar ese literal y también la constante `CABECERA`, que se compara con `equals` contra la primera línea (si el encabezado viene con `;`, tiene que coincidir). El `-1` de `split` conserva los campos vacíos del final. Sin él, una fila como `srv-1,R,5,5,` perdería la última columna y cambiaría el conteo de columnas.

Pregunta 4
Una respuesta

Preparcial · Cloud Center

De práctica

Para la fila `srv-0012,US-EAST-1,97,16310,CRITICO` (CPU y memoria dentro de sus rangos), ¿qué ocurre?

```
int indiceCarga = (int) ((usoCpu * 0.7) + (usoMemoria * 0.3));
// (int)(67.9 + 4893.0) = 4960
return new Servidor(tokens[0], tokens[1], indiceCarga);   // exige 1 <= indice <= 70
```

Seleccione una:

- a. Se procesa normalmente porque CPU y memoria son válidos
- b. Lanza IllegalArgumentException porque el índice calculado (4960) queda fuera de 1–70, y la fila queda inválida ✓
- c. Se descarta como si fuera ESTABLE
- d. Lanza NumberFormatException al convertir la memoria

La respuesta correcta es: Lanza IllegalArgumentException porque el índice calculado (4960) queda fuera de 1–70, y la fila queda inválida

Por qué: Cada campo pasa su validación, pero el valor **derivado** no: el constructor tira "El indice de carga es un valor fuera del rango permitido" y el parser la registra como inválida. Es la misma trampa de la pregunta 2 del parcial real.

Pregunta 5
Una respuesta

Preparcial · Cloud Center

De práctica

Una fila trae estado ESTABLE pero datos absurdos: `srv-0099,ZZ,999,999999,ESTABLE`. ¿Qué hace el parser?

Seleccione una:

- ◉ a. La descarta sin excepción: ESTABLE se revisa y se hace continue antes de validar CPU y memoria ✓
- ○ b. Lanza IllegalArgumentException por el CPU fuera de rango
- ○ c. La marca como inválida por memoria fuera de rango
- ○ d. Lanza NumberFormatException

La respuesta correcta es: La descarta sin excepción: ESTABLE se revisa y se hace continue antes de validar CPU y memoria

Por qué: El chequeo de ESTABLE va antes de `desdeCampos`, que es donde se validan CPU y memoria. Por eso el preenunciado dice "con ancho correcto, ESTABLE se descarta sin validar los restantes datos". Si la fila tuviera 4 o 6 campos, sí sería inválida, porque el ancho se controla primero.

Pregunta 6
Desarrollo

Preparcial · Cloud Center

De práctica

En `leer()` se hace `leidas++` y luego `int numero = leidas + 1`. ¿Por qué se suma 1?

Respuesta modelo: `leidas` cuenta solo filas de datos, pero el encabezado ocupa la línea física 1 y no cuenta como leída. La primera fila de datos está en la línea 2: cuando `leidas` vale 1, `numero` vale 2. Así los mensajes muestran la línea real del archivo, por ejemplo "Linea 7: uso del cpu esta fuera de rango", que es lo que controla el test.

Pregunta 7
Una respuesta

Preparcial · Cloud Center

De práctica

En `panorama()` el resultado de `totalesPorRegion()` (un `groupingBy`) se envuelve en `new TreeMap<>(...)`. ¿Para qué?

Seleccione una:

- ◉ a. Para que las regiones salgan siempre en el mismo orden (alfabético), porque el Map de groupingBy no garantiza orden ✓
- ○ b. Es redundante: groupingBy ya devuelve un TreeMap
- ○ c. Para eliminar regiones duplicadas
- ○ d. Para convertir los valores Integer en String

La respuesta correcta es: Para que las regiones salgan siempre en el mismo orden (alfabético), porque el Map de groupingBy no garantiza orden

Por qué: `groupingBy` devuelve un `HashMap`, que no garantiza ningún orden. `TreeMap` ordena por clave: "{SA-EAST-1=5400, US-EAST-1=6790, US-WEST-2=2980}". Si querés que el orden venga ya desde el colector: `groupingBy(clave, TreeMap::new, downstream)`.

Pregunta 8
Una respuesta

Preparcial · Cloud Center

De práctica

¿Qué recibe `Cloud.filtrar()` y qué da `cloud.filtrar(s -> s.getIndiceCarga() < 50).size()` con `datos.csv`?

```
public List<Servidor> filtrar(Predicate<Servidor> criterio) {
    return this.servidores.stream().filter(criterio).toList();
}
```

Seleccione una:

- ◉ a. Un Predicate<Servidor> (boolean test(T)); da 16 ✓
- ○ b. Un Function<Servidor,Boolean>; da 30
- ○ c. Un Consumer<Servidor>; da 0
- ○ d. Un Predicate<Servidor>; da 60

La respuesta correcta es: Un Predicate<Servidor> (boolean test(T)); da 16

Por qué: `Predicate<T>` es la interfaz funcional de una condición booleana. 16 de los 30 servidores procesados tienen índice menor a 50 (lo imprime el programa). Además, `toList()` devuelve una lista inmodificable.

Pregunta 9
Desarrollo

Preparcial · Cloud Center

De práctica

`costoRiesgoOperativo()` calcula `(INGRESO_PROMEDIO * indiceCarga) / 100`: multiplica primero y divide después. ¿Por qué conviene ese orden en aritmética entera? ¿Cambiaría algo acá si fuera `(INGRESO_PROMEDIO / 100) * indiceCarga`?

Respuesta modelo: En enteros, dividir primero trunca y ese error se arrastra; por eso se multiplica antes. Acá no cambia nada porque 1000/100 = 10 es exacto. **Pero en el parcial sí cambia**: con ingresos 1110 e índice 59, `(1110 * 59) / 100` = 654, en cambio `(1110 / 100) * 59` = 11·59 = 649. Eso hace fallar el test del profe.

Pregunta 10
Una respuesta

Preparcial · Cloud Center

De práctica

¿Cuál es el índice de carga de srv-0044 (CPU 67, memoria 13)?

```
int indiceCarga = (int) ((usoCpu * 0.7) + (usoMemoria * 0.3));
// 46.9 + 3.9 = 50.8
```

Seleccione una:

- ◉ a. 50 ✓
- ○ b. 51
- ○ c. 50.8
- ○ d. No compila porque mezcla int y double

La respuesta correcta es: 50

Por qué: El cast `(int)` **trunca**, no redondea: 50,8 queda en 50. Es un caso testigo del preenunciado. Compila porque el cast explícito convierte el double a int.

Pregunta 11
Una respuesta

Preparcial · Cloud Center

De práctica

¿Qué excepción sale si el archivo no existe, y cuál si la primera línea no es el encabezado esperado?

Seleccione una:

- ◉ a. IOException / IllegalArgumentException ✓
- ○ b. FileNotFoundException / RuntimeException genérica
- ○ c. IllegalArgumentException / IOException
- ○ d. IOException en los dos casos

La respuesta correcta es: IOException / IllegalArgumentException

Por qué: `Files.newBufferedReader` propaga `IOException` (el método la declara con `throws`). El encabezado incorrecto tira `IllegalArgumentException("encabezado incorrecto")` y corta la carga. Lo prueba `soloCabeceraYErroresDeArchivo`.

Pregunta 12
Una respuesta

Preparcial · Cloud Center

De práctica

Se procesa este archivo. ¿Cuántas leídas, procesadas, descartadas e inválidas hay?

```
id,region,porcentajeCPU,usoMemoria,estado
S2,S1,101,9,ALERTA
s7,U2,54,40,ALERTA
S7,U2,20,19,ESTABLE
```

Seleccione una:

- ○ a. 3 / 2 / 1 / 0
- ◉ b. 3 / 1 / 1 / 1 ✓
- ○ c. 3 / 0 / 1 / 2
- ○ d. 4 / 1 / 1 / 2

La respuesta correcta es: 3 / 1 / 1 / 1

Por qué: S2: CPU 101 > 100 → inválida. s7: índice (int)(37,8 + 12) = 49 → procesada. S7: ESTABLE → descartada. El encabezado no cuenta como leída. Es el test `estructuraYContinuacion`.

Pregunta 13
Desarrollo

Preparcial · Cloud Center

De práctica

¿Por qué el try/catch de cada fila está **dentro** del while y no rodeando todo `leer()`?

Respuesta modelo: Porque una fila mala no tiene que cortar la carga: se registra el error con su número de línea y se sigue con la próxima. Si el try envolviera todo el método, la primera excepción sacaría del while y se perderían todas las filas siguientes. Fijate que se captura `IllegalArgumentException`, que también atrapa `NumberFormatException` (es subclase), y que la `IOException` no se captura: un fallo de lectura sí aborta.

Pregunta 14
Una respuesta

Preparcial · Cloud Center

De práctica

Con `datos.csv`, ¿qué imprime exactamente `colector.panorama()`?

Seleccione una:

- ◉ a. Total: 15170 | Por Region: {SA-EAST-1=5400, US-EAST-1=6790, US-WEST-2=2980} | indice menores a 50: 16 ✓
- ○ b. Total: 30 | Por Region: {} | indice menores a 50: 60
- ○ c. Leídas: 60 | procesadas: 30 | descartadas: 21 | inválidas: 9 | objetos: 30
- ○ d. Total: 15170 | Por Region: {US-WEST-2=2980, US-EAST-1=6790, SA-EAST-1=5400} | indice menores a 50: 16

La respuesta correcta es: Total: 15170 | Por Region: {SA-EAST-1=5400, US-EAST-1=6790, US-WEST-2=2980} | indice menores a 50: 16

Por qué: La c es la salida de `resultado.informe()`, no de `panorama()`. La d tiene los números bien pero el orden mal: con `TreeMap` las regiones salen en orden alfabético.

3 Stream API y programación funcional (Cuestionario 05)

Lo que más se repite en los cuestionarios. Fijate siempre si la opción dice **flujo** o **lista**: las operaciones intermedias devuelven otro flujo.

Pregunta 15
Varias respuestas
Cuestionario 05 · también en el simulacro

Seleccione la/las opción/es correcta/as referido a Stream API

Seleccione una o más de una:

- ☑ a. Sobre la secuencia de datos se introducen operaciones que manipulan cada dato individualmente. ✓
- ☐ b. Stream mantiene todos los datos almacenados en memoria y los recorre.
- ☑ c. Una de las razones por las que se usa stream es para evitar recorrer arrays. ✓

Las respuestas correctas son: Sobre la secuencia de datos se introducen operaciones que manipulan cada dato individualmente., Una de las razones por las que se usa stream es para evitar recorrer arrays.

Por qué: El apunte 10 lo dice casi textual: "sobre esa secuencia el programador introduce operaciones que manipulen cada dato individualmente, **eliminándolo de la responsabilidad de recorrerlos**". Por eso la c también va. La b es lo que hacen las **colecciones**, no los streams.

Ojo: La guía de repaso que circula marca solo la primera. Pero en los dos intentos del cuestionario, quien marcó solo esa sacó 0,5 de 1: falta otra correcta, y la única posible es la c.

Pregunta 16
Varias respuestas
Cuestionario 05

Seleccione la/las opción/es correcta/as referido a Stream API

Seleccione una o más de una:

- ☑ a. Un stream es una secuencia de datos que van transcurriendo sin estar almacenados todos juntos. ✓
- ☑ b. No recorre los datos inmediatamente, las operaciones intermedias son perezosas (lazy) y esperan a la operación final para ejecutarse. ✓
- ☐ c. Stream procesa todos los datos al mismo tiempo

Las respuestas correctas son: Un stream es una secuencia de datos que van transcurriendo sin estar almacenados todos juntos., No recorre los datos inmediatamente, las operaciones intermedias son perezosas (lazy) y esperan a la operación final para ejecutarse.

Por qué: Apunte 10: "Las operaciones intermedias no recorren el conjunto de datos ni ejecutan nada hasta que no se ejecuta una operación terminal".

Pregunta 17
Una respuesta
Cuestionario 05

¿Cómo se denominan los métodos o funciones de orden superior?

Seleccione una:

- ◉ a. Son los métodos que poseen parámetros o retorno de tipo función ✓
- ○ b. Son los métodos que poseen parámetros o retorno de tipo primitivo
- ○ c. Son los métodos que solo retornan tipo función y poseen parámetros tipo primitivo

La respuesta correcta es: Son los métodos que poseen parámetros o retorno de tipo función

Por qué: Reciben y/o devuelven funciones. `filter`, `map` y `sorted` son de orden superior porque reciben una lambda.

Pregunta 18
Verdadero / falso
Cuestionario 05

La operación terminal forEach recibe como parámetro el dato y no retorna nada

Seleccione una:

- ◉ a. Verdadero ✓
- ○ b. Falso

La respuesta correcta es: Verdadero

Por qué: `forEach` es `void`. También sale al revés ("...y retorna cada uno de los elementos del flujo"): ahí es **Falso**.

Pregunta 19
Verdadero / falso
Cuestionario 05

No hay que programar una clase nueva por cada expresión lambda creada

Seleccione una:

- ◉ a. Verdadero ✓
- ○ b. Falso

La respuesta correcta es: Verdadero

Por qué: La lambda implementa la interfaz funcional sin declarar una clase. También sale al revés ("Hay que programar una clase nueva..."): ahí es **Falso**.

Pregunta 20
Verdadero / falso
Cuestionario 05

Limit recibe un número entero y genera una nueva lista que contiene como máximo la cantidad indicada

Seleccione una:

- ○ a. Verdadero
- ◉ b. Falso ✓

La respuesta correcta es: Falso

Por qué: `limit` genera un nuevo **flujo**, no una lista. La misma frase con "flujo" sale como otra pregunta, y ahí es **Verdadero**. Fijate siempre si dice lista o flujo.

Pregunta 21
Una respuesta
Cuestionario 05

Seleccione las opciones correctas respecto a la operación intermedia map?

Seleccione una:

- ○ a. Recibe como parámetro una instancia de la interfaz funcional Function y mapea con clave → valor una serie de elementos
- ○ b. Retorna un nuevo flujo con los mismos datos del flujo original pero eliminando los repetidos
- ◉ c. Recibe como parámetro una instancia de la interfaz funcional Function y genera un nuevo valor a partir de cada uno de los elementos del flujo ✓

La respuesta correcta es: Recibe como parámetro una instancia de la interfaz funcional Function y genera un nuevo valor a partir de cada uno de los elementos del flujo

Por qué: `map` transforma cada elemento en otro (1 a 1). No arma pares clave-valor (eso es un Map) ni elimina repetidos (eso es `distinct`).

Pregunta 22
Una respuesta

Cuestionario 05

Seleccione la opción correcta respecto a las operaciones terminales

Seleccione una:

- ○ a. Toda manipulación de flujo puede no tener una operación terminal
- ○ b. Retornan un nuevo flujo
- ◉ c. Recolectan todos los elementos de un flujo y los procesan ✓

La respuesta correcta es: Recolectan todos los elementos de un flujo y los procesan

Por qué: Apunte 10: "toda manipulación de un flujo debe finalizar siempre con una operación terminal". Las que devuelven otro flujo son las intermedias.

Pregunta 23
Una respuesta

Cuestionario 05

La operación intermedia llamada sorted retorna un nuevo flujo con los mismos elementos que el original pero

Seleccione una:

- ○ a. desordenados
- ○ b. limitados
- ◉ c. ordenados ✓

La respuesta correcta es: ordenados

Por qué: Por orden natural o con el `Comparator` que se le pase.

Pregunta 24
Verdadero / falso

Cuestionario 05 · también en el simulacro

Con respecto a la operación collector(): Su función principal es acumular, combinar y finalizar los datos procesados en el stream, permitiendo producir colecciones, resúmenes o agrupaciones

Seleccione una:

- ◉ a. Verdadero ✓
- ○ b. Falso

La respuesta correcta es: Verdadero

Por qué: Es la definición de `collect` con `Collectors`: `toList`, `toMap`, `groupingBy`, `counting`, `averagingInt` …

Pregunta 25
Varias respuestas
Cuestionario 05

¿Cómo funcionan las expresiones Lambda?

Seleccione una o más de una:

☑ a. Ofrecen la posibilidad de escribir únicamente el bloque de código del método directamente en el lugar de la declaración de la variable, haciendo automáticamente la sobreescritura del método. ✓

☐ b. Ofrecen la posibilidad de escribir únicamente el bloque de código del método directamente en el lugar de la declaración de la variable, aunque se tenga que hacer la sobreescritura del método de forma manual.

☑ c. La forma de escritura de una expresión lambda es [parámetros]→{cuerpo} ✓

Las respuestas correctas son: Ofrecen la posibilidad de escribir únicamente el bloque de código del método directamente en el lugar de la declaración de la variable, haciendo automáticamente la sobreescritura del método., La forma de escritura de una expresión lambda es [parámetros]→{cuerpo}

Por qué: La lambda implementa sola el único método abstracto de la interfaz funcional. El campus da por buena la c aunque use corchetes. En Java se escribe con paréntesis: `(a, b) -> { ... }`.

Pregunta 26
Una respuesta
Cuestionario 05

Seleccione la opción correcta respecto a los Collectors

Seleccione una:

○ a. Es una clase que define cómo mostrar los elementos de un flujo.

◉ b. Es un objeto que define cómo transformar los elementos de un flujo en un resultado final ✓

○ c. La clase Collectors no provee implementaciones de las colecciones y hay que sobreescribirlas.

La respuesta correcta es: Es un objeto que define cómo transformar los elementos de un flujo en un resultado final

Por qué: `Collectors` ya trae las implementaciones listas para usar.

Pregunta 27
Una respuesta
Cuestionario 05

Cómo se escribe correctamente un stream collector habiendo definido previamente una Lista llamada lista.

Seleccione una:

◉ a. List lista = nombres.stream().collect(Collectors.toList()); ✓

○ b. List lista = nombres.collect(Collectors.toList());

○ c. List lista = nombres.stream(collect(Collectors.toList()));

La respuesta correcta es: List lista = nombres.stream().collect(Collectors.toList());

Por qué: Primero `.stream()` y después `.collect(...)`, encadenados. Una `List` no tiene `collect`.

Pregunta 28
Una respuesta
Cuestionario 05

La operación terminal count retorna la ___ de elementos del flujo

Seleccione una:

◉ a. cantidad ✓

○ b. lista

La respuesta correcta es: cantidad

Por qué: Devuelve un `long`.

Pregunta 29
Varias respuestas
Cuestionario 05

Seleccione las opciones correctas respecto a la operación intermedia filter?

Seleccione una o más de una:

☑ a. Se escribe de la siguiente manera arrayList.stream().filter(x-> x%2==0) ✓

☐ b. Recibe como parámetro una función lambda que recibe un parámetro y retorna una lista

☑ c. Recibe como parámetro una función lambda que recibe un parámetro y retorna un boolean. ✓

Las respuestas correctas son: Se escribe de la siguiente manera arrayList.stream().filter(x-> x%2==0), Recibe como parámetro una función lambda que recibe un parámetro y retorna un boolean.

Por qué: `filter` recibe un `Predicate`: un elemento entra, sale un boolean.

Pregunta 30
Una respuesta
Cuestionario 05

¿Para qué sirve la anotación @FunctionalInterface?

Seleccione una:

◉ a. Para establecer a una interfaz como funcional y asegurarse de que tenga solo un método. ✓

○ b. Para sobrecargar un método funcional

○ c. Para sobreescribir un método

La respuesta correcta es: Para establecer a una interfaz como funcional y asegurarse de que tenga solo un método.

Por qué: El compilador da error si la interfaz no tiene exactamente un método abstracto.

4 Teoría: backend, Java, POO, Maven, excepciones, JUnit y CSV

Las del cuestionario del simulacro y las “preguntas posibles” son del campus. Las marcadas **De práctica** son de los apuntes, para repasar lo que no salió en los cuestionarios.

Pregunta 31
Una respuesta
Simulacro (campus)

¿Qué implica cuando hablamos de “Manejo de la lógica de negocio” en el desarrollo backend?

Seleccione una:

- ◉ a. Todas son correctas. ✓
- ○ b. Implica la implementación y gestión de la lógica de negocio de una aplicación.
- ○ c. Ninguna es correcta.
- ○ d. Implica cómo se llevarán a cabo las operaciones.
- ○ e. Implica definir las reglas y procesos que definen cómo funcionará el sistema.

La respuesta correcta es: Todas son correctas.

Por qué: El apunte 01 lo define en una sola frase que junta b, d y e: “implica la implementación y gestión de la lógica de negocio de una aplicación, es decir, las reglas y procesos que definen cómo funcionará el sistema y cómo se llevarán a cabo las operaciones”. En el Cuestionario 01 aparece lo mismo con los otros desafíos: escalabilidad, seguridad, rendimiento e integración con sistemas externos.

Pregunta 32
Una respuesta
Preguntas posibles · Cuestionario 01

¿Qué responsabilidad principal tiene la subcapa de servicio en una aplicación backend modular?

Seleccione una:

- ○ a. Ejecutar las validaciones de formularios del frontend.
- ○ b. Gestionar las configuraciones del servidor web.
- ○ c. Generar las vistas HTML enviadas al cliente.
- ◉ d. Implementar la lógica de negocio asociada a la funcionalidad. ✓

La respuesta correcta es: Implementar la lógica de negocio asociada a la funcionalidad.

Por qué: Servicio = lógica de negocio. En el proyecto, los cálculos están en `Servidor` y `Cloud`, no en `App`.

Pregunta 33
Una respuesta

Preguntas posibles

La liquidación recibe altas desde una web, una aplicación de mostrador y una importación CSV ejecutada en el backend. La web ya calcula el neto y muestra una comisión estimada. Al cambiar una tarifa, los tres caminos deben aceptar los mismos montos y registrar la misma comisión, incluso si un cliente todavía usa la tarifa anterior. ¿Qué organización satisface mejor ese requisito?

Seleccione una:

- ◉ a. Concentrar validación y cálculo en una operación del backend utilizada por las altas de ambos clientes y por el importador; las pantallas pueden anticipar controles y mostrar estimaciones. ✓
- ○ b. Validar y calcular en cada endpoint del backend y repetir esas reglas en el importador; mantener iguales las copias mediante una especificación común.
- ○ c. Validar los montos en una operación compartida del backend, pero registrar la comisión calculada por cada cliente o importador para conservar lo que vio el operador.
- ○ d. Publicar montos y tarifas en la API para que cada cliente valide y calcule; el backend registra los valores recibidos y el importador aplica su propia copia de las reglas.

La respuesta correcta es: Concentrar validación y cálculo en una operación del backend utilizada por las altas de ambos clientes y por el importador; las pantallas pueden anticipar controles y mostrar estimaciones.

Por qué: Una sola fuente de verdad: si la regla está copiada en varios lugares, cuando cambia la tarifa alguna copia queda vieja. El cliente puede mostrar estimaciones, pero lo que vale es lo que calcula el backend. Salió dos veces con las opciones en otro orden.

Pregunta 34
Una respuesta

Simulacro (campus)

El sistema de juegos expone un catálogo mediante un endpoint `GET /juegos`. Actualmente, los juegos se cargan desde el CSV a una lista en memoria al iniciar el servidor. Se requiere que si un administrador agrega un juego manualmente, este persista tras un reinicio del servidor y siga disponible para la App móvil. ¿Qué cambio técnico cumple este objetivo manteniendo el contrato de la API?

Seleccione una:

- ○ a. Hacer que la App móvil guarde una copia local del catálogo y la envíe al backend cada vez que el servidor se reinicie.
- ◉ b. Almacenar los juegos en un sistema de persistencia duradera (como una base de datos) en el backend y recuperarlos al iniciar; mantener el endpoint de la API idéntico. ✓
- ○ c. Aumentar la memoria RAM del servidor para que la lista en memoria no se pierda durante los reinicios programados.
- ○ d. Eliminar el endpoint GET /juegos y permitir que los clientes lean directamente el archivo CSV desde el servidor de archivos.

La respuesta correcta es: Almacenar los juegos en un sistema de persistencia duradera (como una base de datos) en el backend y recuperarlos al iniciar; mantener el endpoint de la API idéntico.

Por qué: La memoria se pierde al reiniciar, haya la RAM que haya. La persistencia es responsabilidad del backend (capa de datos). “Mantener el contrato” quiere decir que el endpoint sigue igual: el cliente no se entera de cómo se guarda.

Pregunta 35
Varias respuestas

Simulacro (campus)

Indica cuál/cuáles es/son características principales del Lenguaje de Programación Java

Seleccione una o más de una:

- ☑ a. Alto nivel ✓
- ☐ b. Todas son características de java
- ☐ c. Solo compilado
- ☐ d. Dependiente de la plataforma
- ☑ e. Orientado a objetos ✓
- ☐ f. Solo interpretado
- ☐ g. Ninguna es característica de Java

Las respuestas correctas son: Alto nivel, Orientado a objetos

Por qué: Apunte 02: Java es de **alto nivel**, **orientado a objetos** y "compilado **e** interpretado": se compila a bytecode y la JVM lo ejecuta. Por eso no es "solo" una cosa ni la otra. Es **independiente** de la plataforma: la JVM es la que depende del sistema operativo.

Pregunta 36
Una respuesta

Preguntas posibles

El lenguaje de Programación Java es

Seleccione una:

- ○ a. Funcional
- ◉ b. Orientado a objetos ✓
- ○ c. Lógico

La respuesta correcta es: Orientado a objetos

Por qué: Desde Java 8 tiene lambdas y streams, pero su paradigma es la POO.

Pregunta 37
Verdadero / falso

Simulacro (campus)

Para sobrecargar un método es necesario cambiar ya sea su cantidad de parámetros, los tipos de dichos parámetros o ambos.

Seleccione una:

- ◉ a. Verdadero ✓
- ○ b. Falso

La respuesta correcta es: Verdadero

Por qué: Apunte 08: se mantiene el nombre y se modifica la lista de parámetros.

Pregunta 38
Verdadero / falso

Preguntas posibles

El compilador distingue entre las diversas sobrecargas de un método por la forma del tipo de retorno de la función

Seleccione una:

- ○ a. Verdadero
- ◉ b. Falso ✓

La respuesta correcta es: Falso

Por qué: Solo por los parámetros (cantidad, tipo, orden). Dos métodos que solo difieren en el retorno no compilan.

Pregunta 39
Verdadero / falso

Simulacro (campus)

Redefinir un método es la acción de volver a definir en una clase derivada un método heredado de la superclase cuando su implementación no es adecuada para la clase derivada.

Seleccione una:

- ◉ a. Verdadero ✓
- ○ b. Falso

La respuesta correcta es: Verdadero

Por qué: Es el `@Override`: misma firma que en la superclase. Con eso funciona el polimorfismo (pregunta 10 del parcial). No confundir: **sobrecargar** = mismo nombre y otros parámetros; **redefinir** = misma firma en la subclase.

Pregunta 40
Una respuesta

Preguntas posibles

¿Cuál es la regla de la herencia múltiple?

Seleccione una:

- ○ a. A nivel gráfico UML no existe más de una flecha que parte desde la clase derivada hacia alguna super clase
- ◉ b. Una clase puede tener solo una super clase directa ✓
- ○ c. Una clase puede tener tantas super clases directas como se desee

La respuesta correcta es: Una clase puede tener solo una super clase directa

Por qué: Java tiene herencia **simple**: un solo `extends`. Sí puede implementar varias interfaces.

Pregunta 41
Verdadero / falso

Preguntas posibles

Al definir el método main, se lo define como static, esto quiere decir que este no puede invocar otros métodos de su misma clase que a su vez no sean estáticos y por eso es que todos los métodos dentro de main deben de ser declarados estáticos

Seleccione una:

- ○ a. Verdadero
- ◉ b. Falso ✓

La respuesta correcta es: Falso

Por qué: Desde `main` se puede usar cualquier método de instancia creando un objeto: `new ParserServidores().leer(archivo)` es justo eso.

Pregunta 42
Una respuesta

Preguntas posibles

Seleccione el modificador al que corresponde la definición. "Esta función de la clase X es accesible para todas las clases externas y por sí misma."

Seleccione una:

- ○ a. default
- ◉ b. public ✓
- ○ c. private
- ○ d. protected

La respuesta correcta es: public

Por qué: `public`: todas. `protected`: el paquete y las subclases. *default* (sin modificador): solo el paquete. `private`: solo la clase.

Pregunta 43
Varias respuestas

Preguntas posibles

Indique cuáles son las ventajas de usar var como tipo de dato

Seleccione una o más de una:

- ☐ a. Reduce la claridad si se usa de manera excesiva o en contextos donde el tipo no es obvio.
- ☑ b. Facilita el mantenimiento del código al cambiar los tipos de variables sin necesidad de modificar las declaraciones de tipo ✓
- ☑ c. Reduce la repetición de tipos verbosos ✓

Las respuestas correctas son: Facilita el mantenimiento del código al cambiar los tipos de variables sin necesidad de modificar las declaraciones de tipo, Reduce la repetición de tipos verbosos

Por qué: La a es una **desventaja**. `var` solo se usa en variables locales con inicialización: el tipo lo infiere el compilador y sigue siendo estático.

Pregunta 44
Varias respuestas

Preguntas posibles

Seleccione las clases de envoltorio (wrapper classes) de cada tipo primitivo

Seleccione una o más de una:

- ☑ a. Byte ✓
- ☐ b. integer
- ☐ c. String
- ☑ d. Integer ✓

Las respuestas correctas son: Byte, Integer

Por qué: Los wrappers son Byte, Short, Integer, Long, Float, Double, Character y Boolean. “integer” en minúscula no existe y String no envuelve a ningún primitivo.

Pregunta 45
Una respuesta

Apunte 03 · Maven

De práctica

¿Qué es un archetype en Maven?

Seleccione una:

- ◉ a. Una plantilla que genera la estructura inicial del proyecto (por ejemplo, maven-archetype-quickstart) ✓
- ○ b. El archivo pom.xml ya compilado
- ○ c. Un plugin que ejecuta los tests
- ○ d. El repositorio remoto donde se guardan las dependencias

La respuesta correcta es: Una plantilla que genera la estructura inicial del proyecto (por ejemplo, maven-archetype-quickstart)

Por qué: `mvn archetype:generate` arma las carpetas `src/main/java`, `src/test/java` y el `pom.xml`.

Pregunta 46
Varias respuestas

Apunte 03 · Maven

De práctica

Para generar un proyecto con `mvn archetype:generate`, ¿qué datos del proyecto son obligatorios?

Seleccione una o más de una:

- ☑ a. groupId ✓
- ☑ b. artifactId ✓
- ☑ c. version ✓
- ☐ d. El nombre del autor

Las respuestas correctas son: groupId, artifactId, version

Por qué: Las "coordenadas" Maven: groupId (organización, al revés: `ar.edu.utn`), artifactId (el proyecto) y version.

Pregunta 47
Una respuesta

Apunte 03 · Maven

De práctica

¿Cuál es el orden de las fases principales del ciclo de vida de Maven?

Seleccione una:

- ◉ a. validate → compile → test → package → install → deploy ✓
- ○ b. compile → validate → package → test → deploy → install
- ○ c. test → compile → validate → install → package → deploy
- ○ d. package → compile → test → validate → deploy → install

La respuesta correcta es: validate → compile → test → package → install → deploy

Por qué: Cada fase ejecuta las anteriores: `mvn package` también valida, compila y corre los tests. `install` copia el jar al repositorio local (`~/.m2`).

Pregunta 48
Desarrollo

Apuntes 03 y 07

De práctica

Al declarar JUnit en el pom se pone `<scope>test</scope>`. ¿Qué efecto tiene y por qué tiene sentido?

Respuesta modelo: La dependencia está disponible solo para compilar y correr los tests (`src/test`). No va en el artefacto final ni en la ejecución real. JUnit sirve para probar durante el desarrollo, y el programa entregado no la necesita. El preenunciado lo dice: "JUnit 5 es la única dependencia y se usa solamente en tests".

Pregunta 49
Una respuesta

Apuntes 03 y 07

De práctica

Según la convención de Maven, ¿dónde van las clases de test y qué implica?

Seleccione una:

- ◉ a. En src/test/java; se compilan para correr los tests y no forman parte del jar ✓
- ○ b. En src/main/java junto con el código; se empaquetan igual
- ○ c. En la raíz del proyecto, fuera de src
- ○ d. En target/test, que se genera a mano

La respuesta correcta es: En src/test/java; se compilan para correr los tests y no forman parte del jar

Por qué: En el parcial había que copiar `tests-nuevos/ar` a `src/test/java/ar`. `target/` lo genera Maven, por eso no se entrega en el ZIP.

Pregunta 50
Una respuesta

Apunte 06 · Arreglos

De práctica

Un arreglo `int[] v = new int[6];` tiene índices válidos de:

Seleccione una:

- ◉ a. 0 a 5 ✓
- ○ b. 1 a 6
- ○ c. 0 a 6
- ○ d. 1 a 5

La respuesta correcta es: 0 a 5

Por qué: De 0 a N−1. `v[6]` tira `ArrayIndexOutOfBoundsException`.

Pregunta 51
Una respuesta

Apunte 06 · Arreglos

De práctica

¿Cómo se obtiene la cantidad de elementos de un arreglo `v`?

Seleccione una:

- ◉ a. v.length (atributo) ✓
- ○ b. v.length() (método)
- ○ c. v.size()
- ○ d. length(v)

La respuesta correcta es: v.length (atributo)

Por qué: En arreglos es `length` sin paréntesis. En String es `length()` y en colecciones `size()`.

Pregunta 52
Una respuesta

Apunte 06 · Excepciones

De práctica

¿Cuáles son excepciones NO chequeadas (unchecked)?

Seleccione una:

- ◉ a. Error y las que heredan de RuntimeException (NullPointerException, IllegalArgumentException...) ✓
- ○ b. Todas las que heredan de Exception
- ○ c. Solo IOException
- ○ d. Ninguna: en Java todas son chequeadas

La respuesta correcta es: Error y las que heredan de RuntimeException (NullPointerException, IllegalArgumentException...)

Por qué: El compilador no obliga a tratarlas. `IOException` es chequeada: por eso `leer()` tiene `throws IOException`. `IllegalArgumentException` y `NumberFormatException` no son chequeadas.

Pregunta 53
Una respuesta

Apunte 06 · Excepciones

De práctica

Si un try puede lanzar IOException y también Exception, ¿en qué orden van los catch?

Seleccione una:

- ◉ a. Primero el más específico (IOException) y después el general (Exception) ✓
- ○ b. No importa el orden
- ○ c. Primero el general y después el específico
- ○ d. Tienen que ir en un único catch(Exception e)

La respuesta correcta es: Primero el más específico (IOException) y después el general (Exception)

Por qué: Si el general va primero, el específico nunca se alcanza y el proyecto **no compila**.

Pregunta 54
Una respuesta

Apunte 06 · Excepciones

De práctica

¿En qué situación un bloque finally NO se ejecuta?

Seleccione una:

- ◉ a. Cuando se llama a System.exit() dentro del try o del catch ✓
- ○ b. Cuando no hay excepción
- ○ c. Cuando la excepción es unchecked
- ○ d. Siempre que haya un catch

La respuesta correcta es: Cuando se llama a System.exit() dentro del try o del catch

Por qué: Fuera de eso, siempre se ejecuta. El try-with-resources (el del parser) cierra el archivo solo, como si fuera un finally.

Pregunta 55
Una respuesta

Apunte 07 · JUnit

De práctica

¿Qué diferencia hay entre @BeforeEach y @BeforeAll?

Seleccione una:

- ◉ a. @BeforeEach corre antes de cada test; @BeforeAll una sola vez antes de todos ✓
- ○ b. Son sinónimos
- ○ c. Es al revés
- ○ d. @BeforeEach es solo para tests parametrizados

La respuesta correcta es: @BeforeEach corre antes de cada test; @BeforeAll una sola vez antes de todos

Por qué: @BeforeAll va en un método `static`.

Pregunta 56
Una respuesta

Apunte 07 · JUnit

De práctica

¿Qué método se usa para comprobar que un código lanza una excepción de un tipo dado?

Seleccione una:

- ◉ a. assertThrows(Clase.class, () → codigo()) ✓
- ○ b. assertEquals(Clase.class, codigo())
- ○ c. assertTrue(codigo() instanceof Clase)
- ○ d. assertNotNull(codigo())

La respuesta correcta es: assertThrows(Clase.class, () → codigo())

Por qué: Así están escritos los tests del preparcial:
`assertThrows(IllegalArgumentException.class, () -> new Servidor(null, "W1", 60))`.

Pregunta 57
Una respuesta

Apunte 07 · JUnit

De práctica

¿Qué ventaja tiene agrupar comprobaciones en `assertAll(...)`?

Seleccione una:

- ◉ a. Ejecuta todas aunque alguna falle y reporta todos los fallos juntos ✓
- ○ b. Es más rápido
- ○ c. Reemplaza a @Test
- ○ d. Ninguna

La respuesta correcta es: Ejecuta todas aunque alguna falle y reporta todos los fallos juntos

Por qué: Con `assertEquals` sueltos, el primero que falla corta el test.

Pregunta 58
Una respuesta

Apunte 11 · CSV

De práctica

¿Cuáles son las dos formas de leer un CSV línea por línea solo con el JDK, sin dependencias externas?

Seleccione una:

- ◉ a. Scanner + split, y BufferedReader (Files.newBufferedReader) + split ✓
- ○ b. OpenCSV CSVReader y CsvToBean
- ○ c. CSVReaderHeaderAware y Jackson CSV
- ○ d. JDBC y ResultSet

La respuesta correcta es: Scanner + split, y BufferedReader (Files.newBufferedReader) + split

Por qué: OpenCSV es una dependencia que se agrega al pom. El preparcial usa BufferedReader y split.

Pregunta 59
Desarrollo

Apunte 11 · CSV

De práctica

¿Qué ventaja da BufferedReader sobre Scanner para leer un CSV?

Respuesta modelo: Da más control sobre el **charset** (en el proyecto, `Files.newBufferedReader(archivo, StandardCharsets.UTF_8)`, para que no se rompan los acentos), sobre el manejo de las excepciones de E/S (`IOException`) y sobre el cierre del recurso con try-with-resources. Scanner es más corto de escribir, pero da menos control.

Pregunta 60
Una respuesta

Apuntes 06 y 11

De práctica

El apunte recomienda hacer `trim()` / `strip()` y controlar vacíos antes de convertir un campo a número. ¿Qué excepción se busca evitar?

Seleccione una:

- ◉ a. NumberFormatException ✓
- ○ b. ClassCastException
- ○ c. ArrayIndexOutOfBoundsException
- ○ d. UnsupportedOperationException

La respuesta correcta es: NumberFormatException

Por qué: `Integer.parseInt(" 1110 ")` falla por los espacios; por eso el parser hace `map(String::strip)`. Un campo vacío también falla (la línea 128 del parcial).

5 Práctica con los proyectos de los simulacros

Las mismas ideas del preparcial (copia defensiva, polimorfismo, parseo y separación de responsabilidades) con otros dominios: créditos (3K1), juegos (3K4) y tarjetas (3K7).

Pregunta 61
Una respuesta

Preguntas posibles · 3K1 fintech

Al analizar la inicialización del mapa `tiposFormateados` mediante el siguiente fragmento de código, ¿cuál afirmación es correcta?

```
Map tiposFormateados = maximaTasaInteresPorTipo().entrySet().stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        e -> String.format("%.2f", e.getValue()),
        (newKey, newValue) -> newKey,
        LinkedHashMap::new));
```

Seleccione una:

- a. El código arrojará una excepción en tiempo de ejecución si el método maximaTasaInteresPorTipo() devuelve claves duplicadas, ya que la función de fusión (merge function) ignora los nuevos valores ingresados.
- b. El código altera directamente el estado interno del mapa devuelto por maximaTasaInteresPorTipo(), formateando sus elementos in-place sin reservar espacio adicional en memoria.
- ◉ c. El código transforma los valores numéricos a cadenas con dos decimales, conserva de forma segura el valor existente en caso de colisión de claves y garantiza que se mantenga el orden de inserción original al estructurar el resultado en un LinkedHashMap. ✓
- d. La expresión (newKey, newValue) → newKey actúa como un filtro que elimina automáticamente cualquier clave que no sea de tipo String durante el proceso de recolección.

La respuesta correcta es: El código transforma los valores numéricos a cadenas con dos decimales, conserva de forma segura el valor existente en caso de colisión de claves y garantiza que se mantenga el orden de inserción original al estructurar el resultado en un LinkedHashMap.

Por qué: `toMap` con 4 argumentos: clave, valor, **función de fusión** (qué hacer si se repite una clave; acá se queda con el primero, aunque el parámetro se llame `newKey`) y la **fábrica** del mapa. Como hay función de fusión, no hay excepción por duplicados. `collect` siempre crea un mapa nuevo y no toca el original.

Pregunta 62
Una respuesta

Preguntas posibles · 3K1 fintech

Asumiendo que se procesaron y almacenaron en la CarteraCreditos 4 SolicitudCredito con la siguiente estructura (solo tipo y score): [("PREMIUM", 65), ("REGULAR", 85), ("PREMIUM", 75), ("PREMIUM", 65)]. ¿Cuál sería la salida esperada del método `maximaTasaInteresPorTipo()`?

```
public double getTasaInteresAnual() {          // en SolicitudCredito
    return TASA_BASE + (100 - this.scoreFinanciero) * 0.8;   // TASA_BASE = 40
}
// CarteraCreditos: groupingBy(tipo, mapping(getTasaInteresAnual,
maxBy(naturalOrder())))
```

Seleccione una:

- a. {REGULAR=76, PREMIUM=52}
- b. {REGULAR=52, PREMIUM=76} ✓
- c. {REGULAR=105, PREMIUM=90}
- d. {REGULAR=118.80, PREMIUM=119.69}

La respuesta correcta es: {REGULAR=52, PREMIUM=76}

Por qué: Con el código del simulacro: REGULAR (score 85) da 40 + 15·0,8 = **52**. En PREMIUM, la tasa **máxima** sale del score **mínimo** (65): 40 + 35·0,8 = 68. La única opción con REGULAR = 52 es la b.

Ojo: La b dice PREMIUM = 76, pero con este código da 68 (76 saldría con un score de 55). Se elige la b por descarte. Lo importante es entender que la tasa máxima corresponde al score más bajo.

Pregunta 63
Desarrollo

Simulacro (campus) · 3K4 juegos

Analice el siguiente fragmento del constructor de ServicioJuegos y explique qué valor devuelve `calcularSumaPuntuacionAjustada()` al final. Justifique comparando qué pasaría si se usara `this.juegos = lista` en lugar de `List.copyOf(lista)`.

```
var lista = new ArrayList();
lista.add(new JuegoMesa("J1", "A", 2020, 1, 4, 60, 8.0, 2.0, "Cat")); // Puntuación
16.0
var servicio = new ServicioJuegos(lista);
lista.clear();
double total = servicio.calcularSumaPuntuacionAjustada();
```

Respuesta modelo: Con `List.copyOf(lista)`, ServicioJuegos guarda su propia copia **antes** del `clear()`, así que **total = 16.0**. Con `this.juegos = lista`, las dos variables apuntan a la misma lista: el `clear()` de afuera también vacía la del servicio y **total = 0.0**. La copia defensiva protege el encapsulamiento. *Salió igual con la fintech: CarteraCreditos con una SolicitudCredito PREMIUM de score 45. Con copyOf el promedio es 40 + 55·0,8 = 84,0; sin copia, la lista queda vacía y `average().orElse(0)` da 0,0. El enunciado de esa versión dice `new CentralEnvios(entrada)`, pero es un error de copiado: quiere decir CarteraCreditos.*

Pregunta 64
Una respuesta

Simulacro (campus) · 3K4 juegos

Suponga que JuegoCooperativo extiende de JuegoMesa y sobrescribe `getPuntuacionAjustada()`. Se ejecuta el siguiente código. ¿Qué valor imprime la consola?

```
List lista = List.of(
    new JuegoMesa("J1", "Normal", 2020, 1, 4, 60, 8.0, 2.5, "E"),          //
Puntuación: 20.0
    new JuegoCooperativo("J2", "Coop", 2021, 2, 5, 90, 8.0, 2.5, "C", 6) // Puntuación:
23.0
);
System.out.println(lista.stream().mapToDouble(JuegoMesa::getPuntuacionAjustada).sum());
```

Seleccione una:

- a. Error de compilación porque JuegoMesa::getPuntuacionAjustada no conoce la dificultad.
- b. 40.0 (ignora el cálculo extra del juego cooperativo)
- c. 46.0 (aplica el cálculo de cooperativo a ambos objetos)
- d. 43.0 ✓

La respuesta correcta es: 43.0

Por qué: Enlace dinámico: aunque la referencia diga `JuegoMesa::`, cada objeto ejecuta su versión. J1 da 20,0 y J2 (cooperativo) da 23,0: suma 43,0. Es lo mismo que `Servidor::costoRiesgoOperativo` con `ServidorConIngresos` en el parcial.

Pregunta 65
Desarrollo

Simulacro (campus) · 3K4 juegos

Explique brevemente (4-6 líneas) qué responsabilidades corresponden a JuegoCooperativo y al ParserJuegos cuando se procesa una fila de tipo “Cooperativo” con una dificultad inválida (ej: 15). Indique qué sucede con el resumen del procesamiento y si el parser continúa con la siguiente fila.

Respuesta modelo: JuegoCooperativo protege su invariante: en su constructor (o en su `desdeCampos`) valida la dificultad y, si es 15, tira `IllegalArgumentException`. El objeto inválido **nunca llega a existir**. El **ParserJuegos** no repite esa regla: captura la excepción dentro del while, registra la línea y el motivo en los errores (sube **inválidas**, no se agrega el juego) y **sigue con la próxima fila**. Así las validaciones del dato quedan “cerca del objeto”, como pide el preenunciado, y el parser se ocupa de leer, clasificar y contar.

Pregunta 66
Una respuesta

Simulacro (campus) · 3K4 juegos

Se procesa un archivo con el siguiente contenido (incluyendo encabezado). El parser está configurado para descartar si el rating es menor a 1.0 y marcar como inválida si hay errores de formato o rating > 10.0. ¿Qué valores contiene el ResultadoParseo?

```
id;nombre;anio;min;max;dur;rating;peso;cat;dificultad
J1;Cascadia;2021;1;4;45;8.4;2.1;Familiar;
J2;Error;2020;1;4;30;ocho;2.0;Estrategia;
J3;Fuera;2022;2;5;60;12.0;3.0;Estrategia;
```

Seleccione una:

- a. Leídas: 3, Procesadas: 2, Inválidas: 1
- b. Leídas: 3, Procesadas: 1, Inválidas: 2 ✓
- c. Leídas: 3, Procesadas: 1, Descartadas: 2
- d. El procesamiento se detiene en J2 y lanza una excepción al Main.

La respuesta correcta es: Leídas: 3, Procesadas: 1, Inválidas: 2

Por qué: J1 tiene rating 8,4 → procesada. J2: "ocho" no es número → inválida (`NumberFormatException`). J3: 12,0 > 10 → inválida. Ninguna tiene rating menor a 1, así que no hay descartadas.

Pregunta 67
Una respuesta

Preguntas posibles · 3K7 tarjetas

Se procesa este archivo del formato original. El Consumo solo se agrega después de construirse correctamente. ¿Qué valores tiene el resumen?

```
id,rubro,importeCentavos,bonificacionCentavos,estado
A,Alimentos,0,0,ANULADO
B,Alimentos,mil,0,APROBADO
C,Alimentos,110000,10000,APROBADO
```

Seleccione una:

- a. Leídas 3, procesadas 1, descartadas 0, inválidas 2: A tiene monto inválido.
- b. La carga termina en B y no hay un resultado de parseo.
- c. Leídas 3, procesadas 2, descartadas 1, inválidas 0: procesar incluye intentar convertir.
- d. Leídas 3, procesadas 1, descartadas 1, inválidas 1: C se incorpora después del error de B. ✓

La respuesta correcta es: Leídas 3, procesadas 1, descartadas 1, inválidas 1: C se incorpora después del error de B.

Por qué: A está ANULADO → descartada sin validar los montos (como ESTABLE en el preparcial). B: "mil" no es número → inválida, y la carga sigue. C es válida → procesada.

Pregunta 68
Una respuesta

Preguntas posibles · 3K4 juegos

Se comparan cuatro formas de procesar el archivo de juegos: 1) Scanner y split(";") con índices manuales. 2) OpenCSV, que maneja la sintaxis pero devuelve String[]. 3) Mapas basados en los nombres de encabezados. 4) Mapeo directo a objetos (Beans) por nombre de columna. ¿Qué mejora fundamental aporta el paso del nivel 2 al nivel 3?

Seleccione una:

- ◉ a. Elimina la dependencia de la posición (índice) de las columnas, permitiendo reordenar el CSV si se mantienen los nombres. ✓
- ○ b. Garantiza que no existan filas con datos nulos o vacíos en el archivo.
- ○ c. Permite procesar archivos con diferentes separadores automáticamente.
- ○ d. Automatiza la conversión de tipos sin intervención del programador.

La respuesta correcta es: Elimina la dependencia de la posición (índice) de las columnas, permitiendo reordenar el CSV si se mantienen los nombres.

Por qué: Con `String[]` se accede por posición (`tokens[2]`); con `Map`, por nombre (`fila.get("rating")`). La conversión de tipos recién la automatiza el nivel 4 (beans).

Pregunta 69
Una respuesta

Preguntas posibles · 3K7 tarjetas

Una sucursal intercambia el orden de importeCentavos y bonificacionCentavos, conservando sus encabezados. Se comparan cuatro estrategias: 1) Scanner y posiciones. 2) OpenCSV con arreglos y posiciones. 3) Mapas encabezado → campo. 4) Beans por nombre de encabezado con conversión a números. Se deben seguir calculando el neto y las reglas de aceptación. ¿Qué análisis es correcto?

Seleccione una:

- ○ a. El paso 3 a 4 permite trabajar con objetos, pero vuelve obligatorio el orden original de columnas: las propiedades numéricas de un bean solo pueden asociarse por posición.
- ◉ b. El paso 1 a 2 delega la interpretación de la sintaxis, pero conserva accesos posicionales; 3 usa nombres y 4 delega además el mapeo y las conversiones configuradas. Calcular el neto y validar las reglas del negocio sigue siendo necesario. ✓
- ○ c. El paso 2 a 3 evita depender del orden; por eso el mapa también convierte los montos a números y garantiza que la bonificación sea menor al importe, sin controles adicionales.
- ○ d. El paso 1 a 2 elimina tanto la separación manual como la dependencia del orden: al devolver un arreglo, OpenCSV identifica qué posición corresponde a cada nombre de columna.

La respuesta correcta es: El paso 1 a 2 delega la interpretación de la sintaxis, pero conserva accesos posicionales; 3 usa nombres y 4 delega además el mapeo y las conversiones configuradas. Calcular el neto y validar las reglas del negocio sigue siendo necesario.

Por qué: Ninguna biblioteca de lectura hace la **lógica de negocio**: el neto y las reglas de aceptación siempre las programás vos. Solo cambia quién separa los campos (2), cómo se los busca (3: por nombre) y quién los convierte (4).